TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 5

Ingress, Gateway API and Namespaces

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 5: Ingress, Gateway API and Namespaces . . . 3

5.1 Why Ingress? . . . 3

5.2 Gateway API - the next generation . . . 4

5.3 Namespaces . . . 5

5.4 Live lab: one door, two apps . . . 6

Class 5 - Top 20 interview questions . . . 8

K8S · CLASS 05 · 60-90 minutes

Ingress, Gateway API and Namespaces

By the end you can route many apps through one entry point using host and path rules, add TLS, and organise a cluster with namespaces, quotas and contexts.

What you will learn today

- Why one LoadBalancer per app does not scale
- Ingress resources and Ingress controllers
- Host-based and path-based routing, TLS termination
- Gateway API - the modern successor
- Namespaces: isolation, DNS, which objects are namespaced
- ResourceQuota and LimitRange
- kubeconfig, clusters, users and contexts
- Lab: route /menu and /orders behind quickbite.local

Part	Topic	Time
1	Story and Ingress concepts	15 min
2	Gateway API	5 min
3	Namespaces, quotas, contexts	20 min
4	Live lab	25 min
5	Recap and interview questions	10 min

5.1 Why Ingress?

Ravi's story

QuickBite exposed menu, orders and payment with three LoadBalancer Services. The cloud bill showed three load balancers, three public IPs and three SSL certificates to renew. Marketing wanted everything under one address: quickbite.com/menu, quickbite.com/orders. Ravi needed one smart front door.

An **Ingress** is a set of HTTP/HTTPS routing rules: "requests for host X and path Y go to Service Z". An **Ingress controller** is the actual reverse proxy (NGINX, Traefik, HAProxy, cloud load balancers) that reads those rules and does the routing. Without a controller, an Ingress object does nothing.

Everyday example

A shopping mall has one main entrance and a directory board: "Food court - level 3, Cinema - level 4, Electronics - level 1". Visitors use one door and are directed to the right shop. The Ingress is the directory board; the Ingress controller is the security guard at the entrance who reads it and points you the right way.

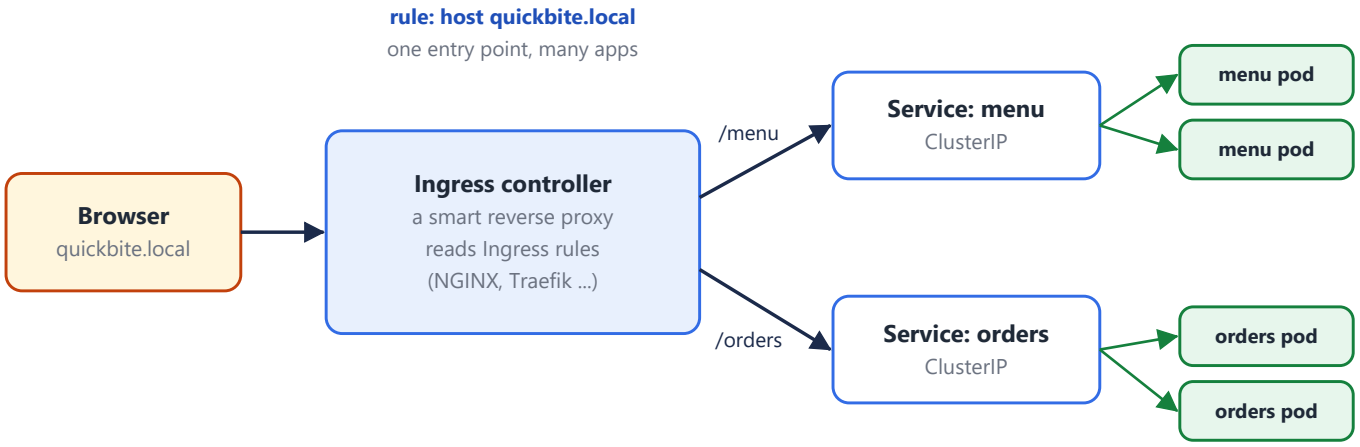


Figure 5.1 - One Ingress controller routes requests to several Services by path

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite
spec:
  ingressClassName: nginx
  tls:                                # optional HTTPS
    - hosts: ["quickbite.local"]
      secretName: quickbite-tls
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /menu
            pathType: Prefix
            backend:
              service: { name: menu, port: { number: 80 } }
          - path: /orders
            pathType: Prefix
            backend:
              service: { name: orders, port: { number: 80 } }
```

Term	Meaning
Path-based routing	`/menu` to one Service, `/orders` to another
Host-based routing	`menu.quickbite.com` and `admin.quickbite.com` to different Services
pathType	`Prefix` (matches /menu/...), `Exact` (only /menu), `ImplementationSpecific`
TLS termination	The controller decrypts HTTPS using a certificate stored in a TLS Secret
IngressClass	Chooses which controller handles this Ingress when several are installed
cert-manager	A popular add-on that obtains and renews free Let's Encrypt certificates automatically

5.2 Gateway API - the next generation

Ingress only covers basic HTTP routing; advanced features ended up in controller-specific annotations. The **Gateway API** is the official successor with richer, portable objects:

- **GatewayClass** - which implementation (like IngressClass).
- **Gateway** - the actual listener (ports, TLS), usually owned by the platform team.
- **HTTPRoute / GRPCRoute / TCPRoute** - routing rules, owned by app teams; supports header matching and weighted traffic splitting for canaries.

Did you know?
The community **ingress-nginx** controller was retired in March 2026 and no longer receives fixes. Ingress objects still work with other controllers, but new projects should prefer the Gateway API or a maintained controller. minikube's ingress addon is still perfectly fine for learning the concepts.

5.3 Namespaces

A **namespace** is a virtual cluster inside a physical cluster. It groups objects, scopes names (two namespaces can each have a Service called `web`), and is the unit for quotas, RBAC permissions and network policies.

Everyday example
One office building, many floors. Finance on floor 2 and HR on floor 3 can both have a room called "Meeting Room A". Each floor has its own budget (quota) and access cards (RBAC), but they share the lifts, power and security (the cluster).

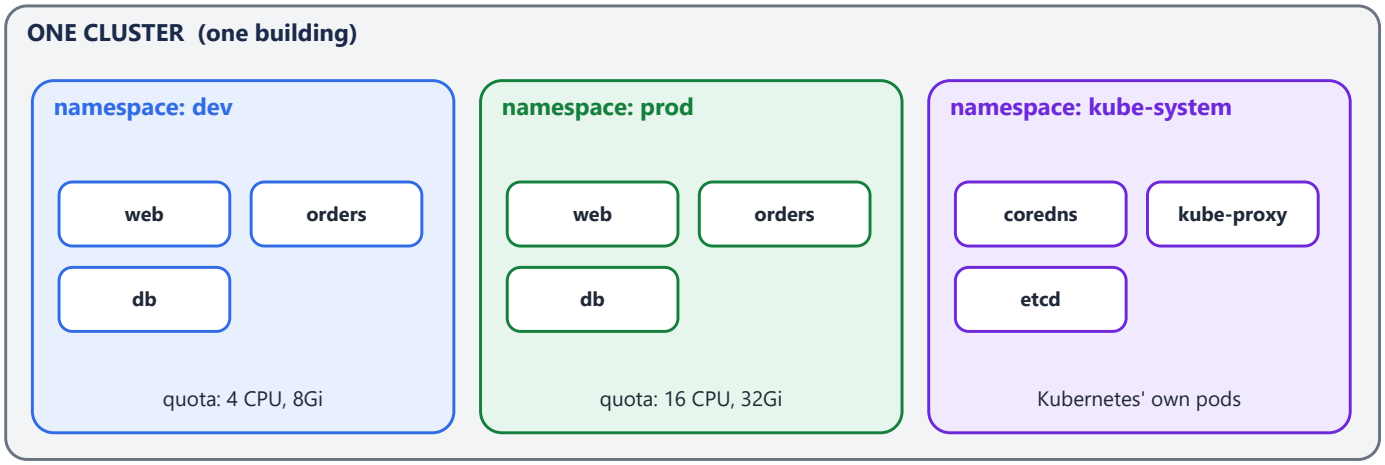


Figure 5.2 - Namespaces divide one cluster into isolated areas

Default namespace	Purpose
default	Where objects go if you do not specify a namespace
kube-system	Kubernetes' own components (CoreDNS, kube-proxy, control plane)
kube-public	Readable by everyone, rarely used
kube-node-lease	Node heartbeat Lease objects for fast failure detection

Not everything is namespaced. Nodes, PersistentVolumes, StorageClasses, ClusterRoles and Namespaces themselves are **cluster-scoped**. Check with `kubectl api-resources --namespaced=false`.

ResourceQuota and LimitRange

quota.yaml

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "2"
    requests.memory: 4Gi
    limits.cpu: "4"
    limits.memory: 8Gi
---
apiVersion: v1
kind: LimitRange
metadata:
  name: dev-defaults
  namespace: dev
spec:
  limits:
    - type: Container
      default:          { cpu: 500m, memory: 256Mi }   # limit if none given
      defaultRequest:   { cpu: 100m, memory: 128Mi }   # request if none given
```

ResourceQuota caps the total a namespace may use. **LimitRange** sets default and min/max values for each container, so pods without resources still get sensible values.

kubeconfig and contexts

kubectl reads `~/.kube/config` (on Windows `C:\Users\<you>\.kube\config`). It lists **clusters** (API server addresses), **users** (credentials) and **contexts** (cluster + user + default namespace). Switching context switches which cluster you control.

```
kubectl config get-contexts
kubectl config current-context
kubectl config use-context minikube
kubectl config set-context --current --namespace=dev   # change default namespace
```

Common mistake
Running `kubectl delete` while your context points at production. Always check `kubectl config current-context` - many engineers show it in their shell prompt.

5.4 Live lab: one door, two apps

PowerShell - part 1

```
minikube addons enable ingress
kubectl get pods -n ingress-nginx          # wait until controller is Running

kubectl create namespace shop
kubectl config set-context --current --namespace=shop

kubectl create deployment menu   --image=hashicorp/http-echo -- /http-echo -text="Menu: biryani"
kubectl create deployment orders --image=hashicorp/http-echo -- /http-echo -text="Orders service OK"
kubectl expose deployment menu   --port=80 --target-port=5678
kubectl expose deployment orders --port=80 --target-port=5678
```

Save the Ingress from section 5.1 as `ingress.yaml`, remove the `tls` block for now, and apply it:

PowerShell - part 2

```
kubectl apply -f ingress.yaml
kubectl get ingress
minikube tunnel                            # separate admin window; keep it open

curl.exe -H "Host: quickbite.local" http://127.0.0.1/menu
curl.exe -H "Host: quickbite.local" http://127.0.0.1/orders
curl.exe -H "Host: quickbite.local" http://127.0.0.1/unknown     # 404 from controller
```

PowerShell - quotas

```
kubectl apply -f quota.yaml            # change namespace: dev -> shop first
kubectl describe quota -n shop
kubectl create deployment big --image=nginx:1.27 --replicas=15
kubectl get deploy big                 # only 10 pods allowed - see why:
kubectl describe rs -l app=big | Select-String quota

kubectl config set-context --current --namespace=default
kubectl delete namespace shop          # deletes everything inside it
```

Try it yourself

- Add `127.0.0.1 quickbite.local` to `C:\Windows\System32\drivers\etc\hosts` (as admin) and open http://quickbite.local/menu in your browser.
- Create a self-signed certificate with OpenSSL, store it with `kubectl create secret tls quickbite-tls --cert=tls.crt --key=tls.key`, add the tls block back and test https.

Class 5 recap

- Ingress = HTTP routing rules; Ingress controller = the proxy that applies them.
- Gateway API = modern, role-oriented successor of Ingress.
- Namespaces scope names, quotas, RBAC and policies.
- ResourceQuota caps totals; LimitRange sets per-container defaults.
- Context = cluster + user + namespace; always check it.

Class 5 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is an Ingress?

An API object that defines HTTP/HTTPS routing rules from outside the cluster to Services, based on host names and URL paths, with optional TLS termination.

Q2. What is an Ingress controller and why is it required?

The component that implements Ingress rules - a reverse proxy such as NGINX, Traefik, HAProxy or a cloud load balancer controller. An Ingress object has no effect unless a controller is running.

Q3. Ingress vs LoadBalancer Service?

A LoadBalancer Service exposes one Service at layer 4 and usually costs one cloud load balancer each. Ingress works at layer 7 and routes many hosts/paths to many Services through one entry point.

Q4. What is path-based vs host-based routing?

Path-based routes by URL path (/menu, /orders) on the same host; host-based routes by hostname (menu.example.com, admin.example.com).

Q5. Explain pathType values.

Prefix matches the path and anything below it split by `/`; Exact matches only the exact path; ImplementationSpecific leaves matching to the controller.

Q6. How do you enable HTTPS with Ingress?

Store the certificate and key in a Secret of type kubernetes.io/tls and reference it under `spec.tls` with the host names. cert-manager can issue and renew certificates automatically.

Q7. What is an IngressClass?

A resource that identifies which controller should handle an Ingress, referenced with `spec.ingressClassName`. It allows multiple controllers in one cluster.

Q8. What is the Gateway API?

The newer Kubernetes networking API that succeeds Ingress. It splits responsibilities into GatewayClass, Gateway and Routes (HTTPRoute, GRPCRoute, etc.), and natively supports header matching, traffic splitting and more protocols.

Q9. What is a namespace?

A logical partition of a cluster that scopes object names and is the boundary for RBAC, quotas, LimitRanges and NetworkPolicies.

Q10. Which namespaces exist by default?

default, kube-system, kube-public and kube-node-lease. Add-ons may create more, like ingress-nginx.

Q11. Are all objects namespaced?

No. Nodes, PersistentVolumes, StorageClasses, Namespaces, ClusterRoles, ClusterRoleBindings and CRDs are cluster-scoped. `kubectl api-resources --namespaced=false` lists them.

Q12. Do namespaces provide network isolation?

No, by default pods in different namespaces can talk to each other. Isolation requires NetworkPolicies and a CNI that enforces them.

Q13. What is a ResourceQuota?

A namespace-level limit on total resource consumption - CPU and memory requests/limits, number of pods, Services, PVCs and so on. New objects that would exceed it are rejected.

Q14. What is a LimitRange?

A namespace policy that sets default requests/limits and minimum/maximum values per container or pod, so workloads without resource settings still get sensible values.

Q15. What happens when you delete a namespace?

All objects inside it are deleted. It stays in Terminating until finalizers complete.

Q16. What is kubeconfig?

A YAML file (default ~/.kube/config) holding clusters, users and contexts that kubectl uses to connect and authenticate. The KUBECONFIG variable can point to other files.

Q17. What is a context in kubectl?

A named combination of cluster, user and default namespace. `kubectl config use-context` switches between them; `set-context --current --namespace=x` changes the default namespace.

Q18. How do you run a command against a namespace without switching?

Add `-n <namespace>` to the command, or `-A`/`--all-namespaces` to list across all.

Q19. Why would you use multiple namespaces instead of multiple clusters?

Namespaces are cheaper and simpler for separating teams or environments with quotas and RBAC. Separate clusters give stronger isolation and are preferred for production vs non-production or strict compliance.

Q20. The Ingress returns 404 or 503. How do you debug?

404: check host header and path rules, pathType and ingressClassName. 503: the backend Service has no ready endpoints - check selectors, ports and pod readiness. Also read the controller pod logs.